1. Ver el problema antes de arreglarlo

Antes de tocar código se reprodujo la falla. TransferenciaService.transferir() hace varias llamadas HTTP independientes (consultar, cargar, abonar, registrar movimientos, notificar) y cada una se confirma por separado en una base distinta.

1.1 Procedimiento

1. Se abrieron las cuentas de Ana (1000) y Beto (500) por el gateway.
2. Se levantó transferencia con una pausa de 20 s entre el cargo y el abono: PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia.
3. Se lanzó una transferencia de 100 de Ana a Beto y, durante la pausa, se apagó cuenta con docker compose stop cuenta.
4. Se encendió cuenta (docker compose start cuenta), se esperó a que se registrara en Eureka y se consultaron los saldos.

Comandos:

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -X POST localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{\"clabe\":\"002180000000000101\",\"titular\":\"Ana\",\"saldoInicial\":1000}'
{"id":24,"clabe":"002180000000000101","titular":"Ana","saldo":1000}
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -X POST localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{\"clabe\":\"002180000000000102\",\"titular\":\"Beto\",\"saldoInicial\":500}'
{"id":25,"clabe":"002180000000000102","titular":"Beto","saldo":500}
```

1.2 Evidencia

Ana con 1000 y Beto con 500 (respuesta de GET /api/cuentas/{clabe}).

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/cuentas/002180000000000101; curl.exe localhost:8080/api/cuentas/002180000000000102
{"id":24,"clabe":"002180000000000101","titular":"Ana","saldo":1000.00}{"id":25,"clabe":"002180000000000102","titular":"Beto","saldo":500.00}
```

Terminal el curl de la transferencia y en otra docker compose stop cuenta. La respuesta puede ser un 504 del gateway (su timeout es de 15 s y la pausa de 20 s).

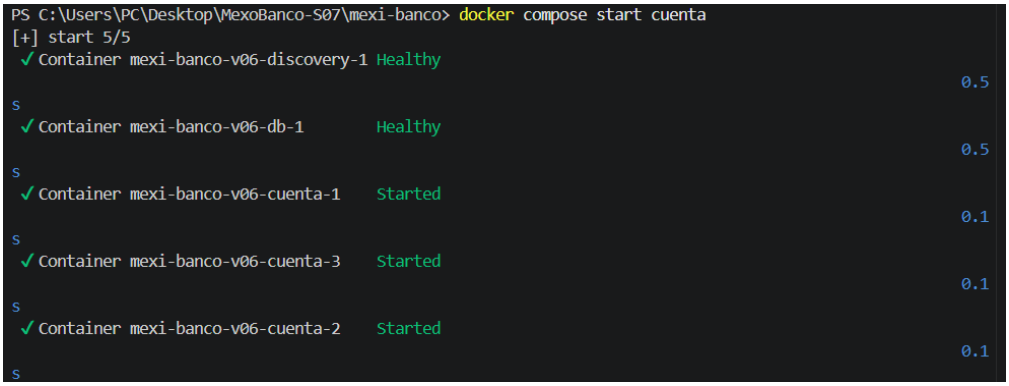
```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -i -X POST localhost:8080/api/transferencias
-H 'Content-Type: application/json' -d '{\"claveOrigen\":\"002180000000000101\",\"claveDestino\":\"0021
80000000000102\",\"monto\":100}'
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json
Content-Length: 140

{"timestamp":"2026-10-10T22:48:39.741+00:00","path":"/api/transferencias","status":504,"error":"Gateway
 Timeout","requestId":"5b77014c-797"}
```

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose stop cuenta
[+] stop 3/3
 ✓ Container mexi-banco-v06-cuenta-3 Stopped                                                    3.4
s
 ✓ Container mexi-banco-v06-cuenta-1 Stopped                                                    3.3
s
 ✓ Container mexi-banco-v06-cuenta-2 Stopped                                                    3.4
s
```

Ana con 900 y Beto con 500. La suma bajó de 1500 a 1400.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/cuentas/0021800000000000101
; curl.exe localhost:8080/api/cuentas/0021800000000000102
{"id":24,"clabe":"0021800000000000101","titular":"Ana","saldo":900.00}{"id":25,"clabe":"0021800000000001
02","titular":"Beto","saldo":500.00}
```



1.3 ¿Por qué pasó?

En el monolito, transferir() tenía @Transactional: cargo y abono se confirmaban juntos o ninguno (atomicidad de ACID). Al partir el sistema, el cargo es una petición HTTP a cuenta que se confirma sola en cuanto responde. Ninguna transacción abarca dos procesos y dos bases, así que cuando el abono falla nadie deshace el cargo.

Además, el cliente recibió un error (504/503) y no sabe si su dinero se movió o no: el resultado de una operación distribuida puede quedar **incierto** para quien la pidió.

2. La saga orquestada

En la versión **orquestada** un componente central, aquí TransferenciaService, decide qué paso sigue y qué compensar. En la **coreografía** cada servicio reacciona a eventos sin un coordinador.

¿Por qué no 2PC? Requiere que todos los participantes bloqueen recursos mientras esperan al coordinador; si el coordinador cae, quedan bloqueados. Con HTTP/REST y bases independientes no tenemos un gestor de transacciones XA, y además reduce la disponibilidad.

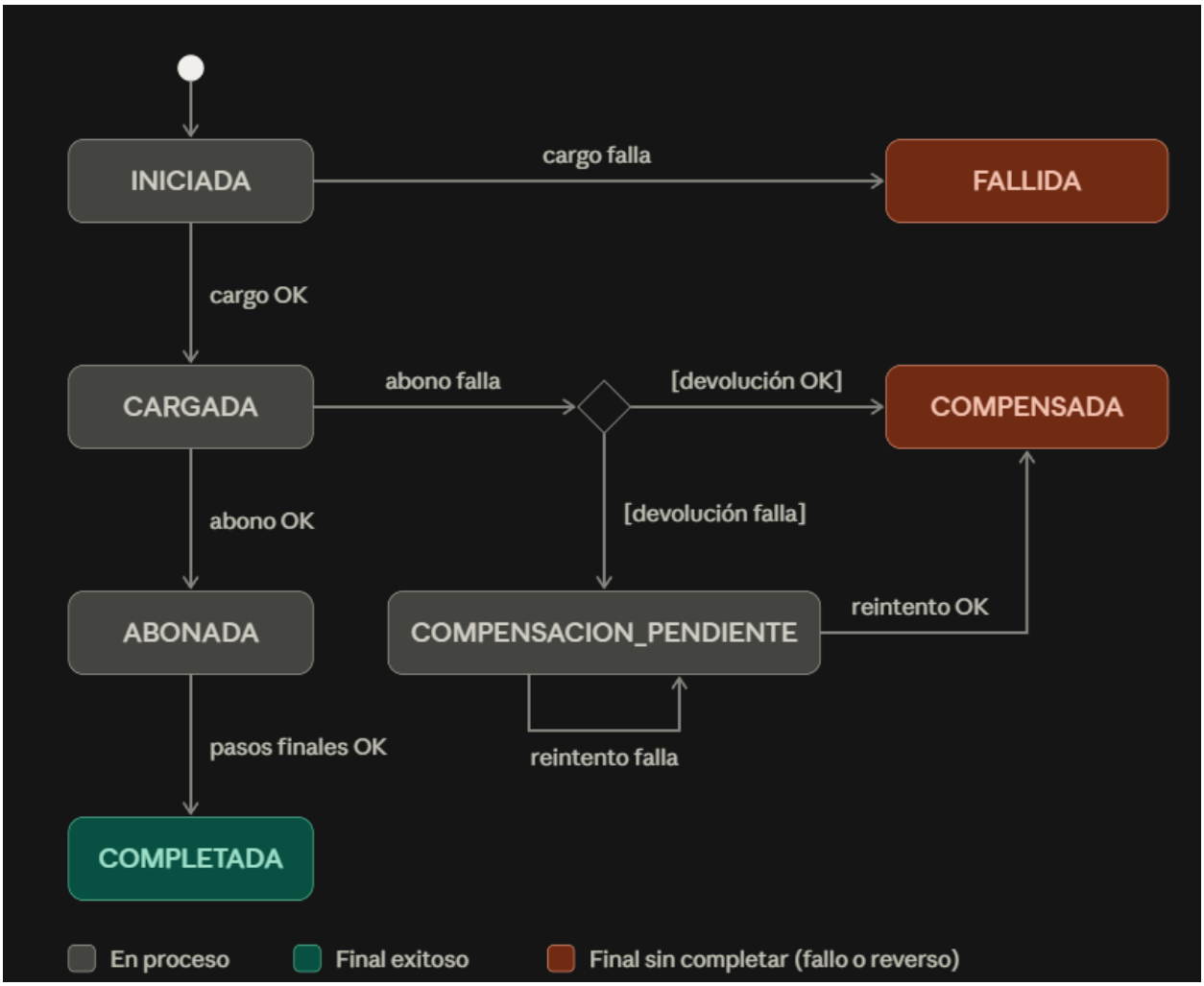
2.1 Pasos de la saga y su compensación

#	Paso (Ti)	Servicio	Compensación (Ci)	Tipo
1	Validar y consultar origen y destino	cuenta (GET)	No necesita: solo lectura	—
2	Guardar la transferencia como INICIADA	transferencia	Marcarla FALLIDA	Local
3	**Cargo** al origen	cuenta (POST /cargos)	**Abono de devolución** al origen + movimiento de devolución	Compensable
4	**Abono** al destino	cuenta (POST /abonos)	— (es el pivote: si sale bien, la saga ya no se compensa)	Pivote
5	Registrar los dos movimientos	movimiento	Se reintenta (no se compensa)	Reintentable
6	Notificar a origen y destino	notificacion	Se reintenta / se tolera	Reintentable
7	Marcar COMPLETADA	transferencia	—	Local

2.2 Estados de la transferencia

Estado	Significado	¿Final?
INICIADA	Se validó y se guardó la fila; todavía no se mueve dinero.	No
CARGADA	El dinero ya salió del origen y aún no llega al destino.	No
ABONADA	El dinero llegó al destino; faltan movimientos y avisos.	No
COMPLETADA	Todos los pasos terminaron bien.	Sí
FALLIDA	Falló antes del cargo (o el cargo fue rechazado): no hay nada que deshacer.	Sí
COMPENSADA	El abono falló y el monto se devolvió al origen, con su movimiento.	Sí
COMPENSACION_PENDIENTE	El abono falló y la devolución también; hay que reintentarla.	No (requiere acción)

2.3 Diagrama de estados



3. El código clave

Todo el cambio quedó dentro del servicio transferencia. Fuera de esa carpeta solo toqué el 5ocker-compose.yml para agregar una variable.

Los estados. Creé el enum EstadoTransferencia con los siete estados de la saga. En Transferencia.java agregué el campo estado y el campo detalle. El estado se guarda como texto para que en la base se lea CARGADA y no un número. Dejé la columna sin la regla de no nula porque la tabla ya tenía transferencias de antes que no tienen estado. En detalle se guarda por qué terminó así la transferencia. Cada transferencia nueva empieza en INICIADA.

El orquestador. En TransferenciaService.java reescribí el método transferir. Primero guardo la transferencia como INICIADA, después hago el cargo y la guardo como CARGADA, luego hago el abono y la guardo como ABONADA, y al final registro los movimientos, mando los avisos y la guardo como COMPLETADA. Al método no le puse @Transactional a propósito, porque así cada guardado se confirma en ese momento. Si el servicio se cae a la mitad, la base dice en qué paso se quedó.

Si falla el cargo, la transferencia queda en FALLIDA y no hay que devolver nada porque el dinero nunca salió. Si falla el abono, el cargo ya pasó y por eso llamo al método compensar. En los dos casos atrapo cualquier RuntimeException y no solo ErrorRemotoException, porque cuando cuenta está apagada el error puede ser de conexión o del balanceador.

La compensación. El método compensar le regresa el dinero a la cuenta de origen con un abono por el mismo monto. Si la devolución sale bien, la transferencia queda en COMPENSADA y registro dos movimientos en el estado de cuenta de Ana, el cargo marcado como revertido y la devolución. Puse los dos porque antes el cargo nunca llegaba a movimiento, y así el saldo y el estado de cuenta sí cuadran. Si la devolución también falla, la transferencia no se queda en CARGADA para siempre, la marco como COMPENSACION_PENDIENTE para saber que ese dinero falta por devolver.

Para la devolución usé el tipo DEPOSITO, que ya existía. El enum de tipos de movimiento está copiado en el servicio movimiento, y si inventaba un tipo nuevo solo en transferencia, movimiento lo iba a rechazar con un 400.

La respuesta al cliente. Creé dos excepciones nuevas, TransferenciaCompensadaException y CompensacionPendienteException, y las agregué en ManejadorDeErrores. Cuando la transferencia se compensa, el cliente recibe un 409 que explica qué falló y cuánto se le devolvió, junto con el número y el estado de la transferencia. Si la compensación quedó pendiente, recibe un 503. Así el cliente sabe qué pasó con su dinero y no le llega un 500 sin explicación.

La falla provocada. Para probar la compensación sin apagar cuenta agregué la variable FALLAR_ABONO_A_CLABE. Si la transferencia va hacia esa CLABE, el abono falla a propósito antes de llamar a cuenta. Si la variable está vacía no hace nada. La agregué en el application.yml de transferencia y en el 5ocker-compose.yml.

Las pruebas. En TransferenciaServiceTest cambié la prueba que mostraba el problema, la que decía que si el abono falla nadie deshace el cargo. Ahora comprueba lo contrario, que si el abono falla el dinero se le regresa al origen. También agregué otra que revisa que si la devolución falla la transferencia quede en COMPENSACION_PENDIENTE. Las 9 pruebas del servicio pasaron.

4. La compensación funcionando

4.1 Procedimiento

1. Se reconstruyó transferencia con la falla activada: FALLAR_ABONO_A_CLABE=002180000000000102 docker compose up –build -d transferencia.
2. Se consultaron los saldos de Ana y Beto.
3. Se lanzó una transferencia de 100 de Ana a Beto, con cuenta encendida.
4. Se consultaron saldos, el estado en la base y el estado de cuenta de Ana.

4.2 Evidencia

Los saldos de Ana y Beto.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/cuentas/002180000000000101; curl.exe localhost:8080/api/cuentas/002180000000000102
{"id":24,"clabe":"002180000000000101","titular":"Ana","saldo":900.00}{"id":25,"clabe":"002180000000000102","titular":"Beto","saldo":500.00}
```

El Problem Details con el código (409/422), el detail que explica la devolución, el id de la transferencia y el estado COMPENSADA.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -i -X POST localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{\"claveOrigen\":\"002180000000000101\
",\"claveDestino\":\"002180000000000102\",\"monto\":100}'
HTTP/1.1 409 Conflict
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sun, 11 Oct 2026 01:11:53 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"El abono a 002180000000000102 fallo (Falla simulada del abono a 002180000000000102 (FALLAR_ABONO_A_CLABE)); se devolv
ieron 100 a 002180000000000101","instance":"/transferencias","transferencia":2,"estado":"COMPENSADA"}
```

Ana con el mismo saldo que antes; Beto sin cambio. La suma se conserva.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/cuentas/002180000000000101; curl.exe localhost:8080/api/cuentas/002180000000000102
{"id":24,"clabe":"002180000000000101","titular":"Ana","saldo":900.00}{"id":25,"clabe":"002180000000000102","titular":"Beto","saldo":500.00}
```

En GET /api/movimientos de Ana aparecen el cargo (-100) y la devolución (+100).
Comando: `curl "localhost:8080/api/movimientos?clabe=002180000000000101"`

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe "localhost:8080/api/movimientos?clabe=002180000000000101"
[{"id":37,"clabeCuenta":"002180000000000101","tipo":"DEPOSITO","monto":100.00,"saldoResultante":900.00,"fecha":"2026-10-11T01:11:53.155583Z","referencia":"devolucion de transferenci
a 2"},{"id":36,"clabeCuenta":"002180000000000101","tipo":"TRANSFERENCIA_ENVIADA","monto":-100.00,"saldoResultante":800.00,"fecha":"2026-10-11T01:11:53.142832Z","referencia":"a 00218
0000000000102 (revertida)"},{"id":35,"clabeCuenta":"002180000000000101","tipo":"DEPOSITO","monto":100.00,"saldoResultante":900.00,"fecha":"2026-10-10T23:36:40.146098Z","referencia":
"devolucion de transferencia 1"},{"id":34,"clabeCuenta":"002180000000000101","tipo":"TRANSFERENCIA_ENVIADA","monto":-100.00,"saldoResultante":800.00,"fecha":"2026-10-10T23:36:40.112
583Z","referencia":"a 002180000000000102 (revertida)"},{"id":32,"clabeCuenta":"002180000000000101","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-10T22
:10:11.747940Z","referencia":"saldo inicial"}]
```

La fila de la transferencia con estado COMPENSADA.

Comando: docker compose exec db psql -U mexibanco -d transferencia -c "select id, clave_origen, clave_destino, monto, estado from transferencia order by id"

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose exec db psql -U mexibanco -d transferencia -c "select id, monto, estado, detalle from transferencia order by id"
 id | monto  |   estado    |                                                
```

Las líneas del log donde se ve INICIADA → CARGADA → falla del abono → COMPENSADA.
Comando: `docker compose logs transferencia --since 2m`

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose logs transferencia --since 10m | Select-String "Transferencia 2"

transferencia-1  | 2026-10-11T01:11:53.088Z  INFO 1 --- [transferencia] [nio-8080-exec-2] m.m.transferencia.TransferenciaService   : Transferencia 2: INICIADA
transferencia-1  | 2026-10-11T01:11:53.106Z  INFO 1 --- [transferencia] [nio-8080-exec-2] m.m.transferencia.TransferenciaService   : Transferencia 2: CARGADA
transferencia-1  | 2026-10-11T01:11:53.111Z  WARN 1 --- [transferencia] [nio-8080-exec-2] m.m.transferencia.TransferenciaService   : Transferencia 2: El abono a 002180000000000102
fallo (Falla simulada del abono a 002180000000000102 (FALLAR_ABONO_A_CLABE)). Compensando
transferencia-1  | 2026-10-11T01:11:53.132Z  INFO 1 --- [transferencia] [nio-8080-exec-2] m.m.transferencia.TransferenciaService   : Transferencia 2: COMPENSADA
```

Una transferencia a una cuenta que no está en FALLAR_ABONO_A_CLABE termina en COMPLETADA.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> $env:FALLAR_ABONO_A_CLABE=""; docker compose up -d transferencia
[+] up 3/3
 ✓ Container mexi-banco-v06-discovery-1     Healthy                                                                                    4.0s
 ✓ Container mexi-banco-v06-db-1            Healthy                                                                                    4.0s
 ✓ Container mexi-banco-v06-transferencia-1 Started                                                                                    4.1s
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose ps transferencia
NAME                            IMAGE                           COMMAND              SERVICE         CREATED          STATUS                            PORTS
mexi-banco-v06-transferencia-1   mexi-banco-v06-transferencia   "java -jar app.jar"   transferencia   11 seconds ago   Up 6 seconds (health: starting)   8080/tcp
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose ps transferencia
NAME                            IMAGE                           COMMAND              SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-transferencia-1   mexi-banco-v06-transferencia   "java -jar app.jar"   transferencia   16 seconds ago   Up 11 seconds (healthy)   8080/tcp
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -i -X POST localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{\"claveOrigen\":\"002180000000000101\
",\"claveDestino\":\"002180000000000102\",\"monto\":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sun, 11 Oct 2026 01:22:55 GMT

{"id":3,"claveOrigen":"002180000000000101","claveDestino":"002180000000000102","monto":50,"fecha":"2026-10-11T01:22:55.602616104Z","estado":"COMPLETADA","detalle":null}
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/cuentas/002180000000000101; curl.exe localhost:8080/api/cuentas/002180000000000102
{"id":24,"clabe":"002180000000000101","titular":"Ana","saldo":850.00}{"id":25,"clabe":"002180000000000102","titular":"Beto","saldo":550.00}
```

Con la variable FALLAR_ABONO_A_CLABE apuntando a la CLABE de Beto, mandé una transferencia de 100 de Ana a Beto. El cargo a Ana sí se hizo, pero el abono a Beto falló a propósito. El orquestador detectó la falla y compensó, le regresó los 100 a Ana y la transferencia quedó en COMPENSADA. El cliente recibió un 409 con un mensaje que explica qué falló y cuánto se devolvió, no un 500 sin explicación.

Los saldos quedaron igual que antes, Ana con 900 y Beto con 500, así que la suma se conservó. En el estado de cuenta de Ana quedaron escritos el cargo marcado como revertido y la devolución, y en los logs se ve el orden de los estados INICIADA, CARGADA, la falla del abono y COMPENSADA.

Al final quité la falla y mandé una transferencia de 50. Esa sí terminó en COMPLETADA, Ana quedó con 850 y Beto con 550, así que la saga no cambió el caso normal.

Ana tiene 900 y no 1000 por los 100 que se perdieron en la Parte 0 con el código viejo. Esa transferencia ni siquiera aparece en la tabla, porque el código viejo guardaba la fila hasta el final y como el abono falló nunca se guardó.

5. Cuando la compensación también falla

5.1 Procedimiento

Se repitió la Parte 0 con el código nuevo: pausa de 20 s entre el cargo y el abono y cuenta apagada durante la pausa. Como cuenta está caída, el abono falla y la devolución también.

El Problem Details que avisa que la devolución está pendiente.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -i -X POST localhost:8080/api/transferencias -H 'Content-Type: appl
ication/json' -d '{\"claveOrigen\":\"00218000000000101\",\"claveDestino\":\"00218000000000102\",\"monto\":100}'
HTTP/1.1 503 Service Unavailable
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Sun, 11 Oct 2026 02:41:02 GMT

{"type":"about:blank","title":"Service Unavailable","status":503,"detail":"El abono a 00218000000000102 fallo (I/O error on
POST request for \"http://cuenta/cuentas/00218000000000102/abonos\": Connection refused) y la devolucion tambien fallo (I/O
error on POST request for \"http://cuenta/cuentas/00218000000000101/abonos\": Connection refused). El monto se devolvera a 0
0218000000000101 al reintentar","instance":"/transferencias","transferencia":4,"estado":"COMPENSACION_PENDIENTE"}
```

La fila con estado COMPENSACION_PENDIENTE (no se quedó en CARGADA).

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose exec db psql -U mexibanco -d transferencia -c "select id, mon
to, estado, detalle from transferencia order by id"
 id | monto  |         estado         |
                                        detalle

----+--------+------------------------+-------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------------------
------------------------------------------------------------
  1 | 100.00 | COMPENSADA             | El abono a 00218000000000102 fallo (Falla simulada del abono a 00218000000000102 (F
ALLAR_ABONO_A_CLABE)); se devolvieron 100 a 00218000000000101
  2 | 100.00 | COMPENSADA             | El abono a 00218000000000102 fallo (Falla simulada del abono a 00218000000000102 (F
ALLAR_ABONO_A_CLABE)); se devolvieron 100 a 00218000000000101
  3 |  50.00 | COMPLETADA             |
  4 | 100.00 | COMPENSACION_PENDIENTE | El abono a 00218000000000102 fallo (I/O error on POST request for "http://cuenta/cue
ntas/00218000000000102/abonos": Connection refused) y la devolucion tambien fallo (I/O error on POST request for "http://cue
nta/cuentas/00218000000000101/abonos": Connection refused)
(4 rows)
```

Ana sigue con el cargo aplicado (el dinero aún no regresa).

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/cuentas/00218000000000101; curl.exe localhost:8
080/api/cuentas/00218000000000102
{"id":24,"clabe":"00218000000000101","titular":"Ana","saldo":750.00}{"id":25,"clabe":"00218000000000102","titular":"Beto","
saldo":550.00}
```

Repetí la Parte 0 con el código nuevo y una pausa de 8 segundos. Mandé una transferencia de 100 de Ana a Beto y durante la pausa apagué cuenta. El cargo a Ana sí se hizo antes de apagarla, después falló el abono a Beto y cuando el orquestador intentó devolverle el dinero a Ana también falló, porque cuenta seguía apagada.

El cliente recibió un 503 que explica que la devolución quedó pendiente. En la base la transferencia quedó en COMPENSACION_PENDIENTE y no en CARGADA. Al encender cuenta, Ana tenía 750 y Beto 550, así que faltan 100. La diferencia con la Parte 0 es que ahora el sistema sabe que faltan, en qué transferencia y por qué, y en la Parte 0 no quedaba ningún registro.

5.2 Cómo se termina una compensación pendiente

Propuesta

Reintento programado: una tarea @Scheduled (con @EnableScheduling) busca cada N segundos las transferencias en COMPENSACION_PENDIENTE y vuelve a ejecutar la compensación.

Espera creciente (backoff exponencial): 2 s, 4 s, 8 s, 16 s… con un máximo y algo de aleatoriedad (jitter), para no golpear a cuenta justo cuando se está levantando.

Límite de intentos: después de K intentos se pasa a un estado como REQUIERE_REVISION y se avisa a una persona (operación manual).

Al arrancar el servicio: también se revisan las filas en CARGADA "viejas", que quedaron así si el propio transferencia se cayó a la mitad.

No lo implementé, es una propuesta. Lo terminaría con una tarea programada que cada cierto tiempo busque en la base las transferencias en COMPENSACION_PENDIENTE y vuelva a intentar la devolución. Para eso ya dejé en el repositorio el método findByEstado.

Entre cada intento esperaría cada vez más tiempo, por ejemplo 2, 4, 8 y 16 segundos, para no saturar a cuenta justo cuando se está levantando. Si después de varios intentos sigue fallando, la pasaría a un estado que pida revisión manual de una persona. También revisaría al arrancar el servicio las transferencias que se quedaron en CARGADA, porque eso pasa si el que se cae a la mitad es el propio transferencia.

5.3 ¿Por qué la compensación debe ser idempotente?

Argumento

Un reintento **no sabe** si el intento anterior falló antes o después de llegar a cuenta. Ejemplo: el abono de devolución sí se aplicó, pero la respuesta se perdió por un timeout. Si se reintenta sin idempotencia, **a Ana se le devuelve el dinero dos veces**: "reintentar sin idempotencia cobra (o paga) dos veces".

Nivel orquestador (implementado): solo se compensa si el estado es CARGADA o COMPENSACION_PENDIENTE, y se marca COMPENSADA en cuanto se confirma. Evita compensar dos veces una transferencia que ya está marcada.

Limitación: eso no cubre el caso de la respuesta perdida, porque el orquestador no sabe que el abono sí ocurrió. Lo correcto es que POST /cuentas/{clabe}/abonos acepte una llave de idempotencia (por ejemplo devolucion-<idTransferencia>), como ya hace SPEI con Idempotency-Key: cuenta guardaría la llave en una columna UNIQUE y, ante un reintento, regresaría el resultado anterior sin abonar otra vez.

6. Circuit breaker en las llamadas a cuenta

6.1 Diseño

Parámetro	Valor
Implementación	A mano
Umbral de fallas para abrir	3
Tiempo en ABIERTO	10 s
Llamadas de prueba en SEMIABIERTO	1
Timeout de conexión / lectura	2 s / 5 s
Qué cuenta como falla	Errores técnicos y 5xx

Hice el circuit breaker a mano en la clase Circuito. Guarda el estado, cuántas fallas seguidas lleva y desde cuándo está abierto. Cada llamada a cuenta pasa primero por el método permitir, que decide si la llamada se hace. Si el circuito está cerrado la deja pasar. Si está abierto y no han pasado los 10 segundos la rechaza de inmediato sin llamar a cuenta. Si ya pasaron, cambia a semiabierto y deja pasar una sola llamada de prueba.

Si la llamada sale bien, el contador vuelve a cero y si estaba en semiabierto se cierra. Si falla, suma una falla y al llegar a tres se abre. Si falla la llamada de prueba se vuelve a abrir y empieza otra espera.

Solo cuento como falla los errores técnicos, cuando no conecta, cuando se acaba el tiempo, cuando el balanceador no encuentra instancias o cuando cuenta responde con un 500. Un 404 o un 422 significa que cuenta sí contestó, así que no abren el circuito. Si contaran, alguien que escribe mal una CLABE tres veces abriría el circuito para todos.

El candado solo protege la decisión de dejar pasar o no y el cambio de estado, no la llamada por la red. Así una llamada lenta a cuenta no bloquea a las demás.

6.2 Evidencia de los tres estados

Una transferencia normal y, si lo expones, el estado CERRADO (endpoint o log).

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/transferencias/circuito
{"servicio":"cuenta","estado":"CERRADO","fallasSeguidas":0}
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -s -o NUL -w "HTTP %{http_code} en %{time_total} s`n" -X POST local
host:8080/api/transferencias -H 'Content-Type: application/json' -d '{\"claveOrigen\":\"002180000000000101\",\"claveDestino\"
:\"002180000000000102\",\"monto\":10}'
HTTP 201 en 0.908732 s
```

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose logs -f transferencia | Select-String "
Circuito"
```

El docker compose stop cuenta y las primeras transferencias, que tardan (timeout) y fallan.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -s -o NUL -w "HTTP %{http_code} en %{time_total} s`n" -X POST local
host:8080/api/transferencias -H 'Content-Type: application/json' -d '{\"claveOrigen\":\"002180000000000101\",\"claveDestino\"
:\"002180000000000102\",\"monto\":10}'
HTTP 201 en 0.908732 s
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose pause cuenta
[+] pause 3/3
 ✓ Container mexi-banco-v06-cuenta-2 Paused                                                                          0.0s
 ✓ Container mexi-banco-v06-cuenta-1 Paused                                                                          0.0s
 ✓ Container mexi-banco-v06-cuenta-3 Paused                                                                          0.0s
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> 1..6 | ForEach-Object { curl.exe -s -o NUL -w "intento $_ -> HTTP %{http_cod
e} en %{time_total} s`n" -X POST localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{\"claveOrigen\":\
"002180000000000101\",\"claveDestino\":\"002180000000000102\",\"monto\":10}' }
intento 1 -> HTTP 503 en 5.028314 s
intento 2 -> HTTP 503 en 5.014759 s
intento 3 -> HTTP 503 en 5.014763 s
intento 4 -> HTTP 503 en 0.007449 s
intento 5 -> HTTP 503 en 0.007577 s
intento 6 -> HTTP 503 en 0.007002 s
```

La línea del log "CERRADO -> ABIERTO" al llegar al umbral.

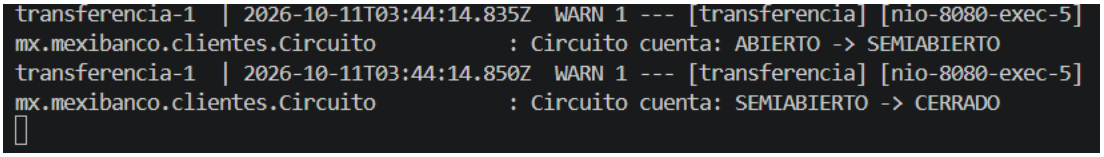
```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose logs -f transferencia | Select-String "
Circuito"

transferencia-1  | 2026-10-11T03:29:49.810Z  WARN 1 --- [transferencia] [nio-8080-exec-5]
mx.mexibanco.clientes.Circuito           : Circuito cuenta: falla 1 de 3 (I/O error on GET request
for "http://cuenta/cuentas/002180000000000101": Read timed out)
transferencia-1  | 2026-10-11T03:29:54.849Z  WARN 1 --- [transferencia] [nio-8080-exec-7]
mx.mexibanco.clientes.Circuito           : Circuito cuenta: falla 2 de 3 (I/O error on GET request
for "http://cuenta/cuentas/002180000000000101": Read timed out)
transferencia-1  | 2026-10-11T03:29:59.881Z  WARN 1 --- [transferencia] [nio-8080-exec-9]
mx.mexibanco.clientes.Circuito           : Circuito cuenta: falla 3 de 3 (I/O error on GET request
for "http://cuenta/cuentas/002180000000000101": Read timed out)
transferencia-1  | 2026-10-11T03:29:59.881Z  WARN 1 --- [transferencia] [nio-8080-exec-9]
mx.mexibanco.clientes.Circuito           : Circuito cuenta: CERRADO -> ABIERTO
transferencia-1  | 2026-10-11T03:29:59.881Z  WARN 1 --- [transferencia] [nio-8080-exec-9]
mx.mexibanco.clientes.Circuito           : Circuito cuenta: abierto por 3 fallas seguidas; se rechaza
todo durante 10000 ms
```

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe localhost:8080/api/transferencias/circuito
{"servicio":"cuenta","estado":"ABIERTO","fallasSeguidas":3}
```

Después de docker compose start cuenta y el tiempo de espera, el log con la llamada de prueba y el cierre del circuito.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose unpause cuenta
[+] unpause 3/3
 ✓ Container mexi-banco-v06-cuenta-3 Unpaused                                                                        0.0s
 ✓ Container mexi-banco-v06-cuenta-1 Unpaused                                                                        0.0s
 ✓ Container mexi-banco-v06-cuenta-2 Unpaused                                                                        0.0s
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> curl.exe -s -o NUL -w "HTTP %{http_code} en %{time_total} s`n" -X POST local
host:8080/api/transferencias -H 'Content-Type: application/json' -d '{\"claveOrigen\":\"002180000000000101\",\"claveDestino\"
:\"002180000000000102\",\"monto\":10}'
HTTP 201 en 0.314003 s
```



Con cuenta congelada, las primeras tres transferencias tardaron unos 5 segundos cada una, que es el tiempo máximo que transferencia espera a cuenta, y respondieron 503. Al llegar a la tercera falla el circuito se abrió y las siguientes respondieron en milisegundos, también con 503, pero sin llamar a cuenta. Así transferencia no se queda colgada ni sigue golpeando a un servicio que no contesta.

Al descongelar cuenta y esperar, la siguiente transferencia pasó como llamada de prueba. El circuito cambió a semiabierto, la prueba salió bien y se cerró. La transferencia terminó en COMPLETADA.

6.3 Interacción con la saga

Si el circuito se abre justo entre el cargo y el abono, la compensación (otro abono a cuenta) **también** choca contra el circuito abierto, así que la transferencia queda en COMPENSACION_PENDIENTE. Esto muestra que el circuit breaker protege al sistema, pero no sustituye al reintento de la compensación: se complementan.

7. Lo que la saga no te da: aislamiento

La saga me da atomicidad eventual, porque al final o se hicieron todos los pasos o se compensaron, pero no me da aislamiento. Cada paso se confirma en su propio servicio en cuanto termina, y desde ese momento cualquiera lo puede ver aunque la saga no haya terminado. Si alguien consulta el saldo de Ana entre el cargo y la compensación, ve el dinero ya descontado aunque después le regrese, como si leyera un dato que al final nunca existió. Si en ese momento Ana intenta otra operación, el banco la puede rechazar por saldo insuficiente aunque el dinero sí sea suyo. Para reducirlo se puede marcar la transferencia como en proceso para que las demás operaciones sepan que ese dinero está apartado, o acomodar los pasos para que lo que más puede fallar vaya antes de mover dinero. Aun así ninguna de estas ideas da el aislamiento completo que tenía el @Transactional del monolito, y ese es el precio de separar el sistema en servicios.